

Description Détaillée du Code - Application CCN 3239

Ce document contient une description détaillée du code de l'application web développée avec NiceGUI pour naviguer dans les articles de la Convention Collective Nationale (CCN) 3239.

1. Structure Générale et Imports

Objectif: Initialiser l'application et importer les dépendances nécessaires.

Détails:

- NiceGUI: Framework pour créer l'interface web en Python.
- sql_manager: Module personnalisé pour gérer les requêtes SQL.
- css: Module personnalisé pour les styles CSS.
- os: Pour gérer les variables d'environnement.
- sqlite3: Pour interagir directement avec la base SQLite.
- re: Pour les expressions régulières.

Fichiers statiques:

- Le dossier /static dans l'URL pointe vers le dossier local static (contient images, PDFs, etc.).

2. Classe AppState: Gestion de l'État Utilisateur

Objectif: Stocker et gérer l'état de l'application par utilisateur.

Explications:

- step: Contrôle le flux de navigation.
- lang: Gère la langue de l'interface (FR/EN).

- choix: Stocke les sélections de l'utilisateur.
- code_metier_affiche: Affiche un libellé lisible pour le métier.
- art_cible: Utilisé pour la recherche directe d'un article.
- annexe_selectionnee: Stocke le numéro de l'annexe en cours de consultation.

Pourquoi une classe ?

- Persistance: Les objets AppState sont créés par utilisateur.
- Réactivité: Les modifications de state déclenchent un rafraîchissement de l'UI.

3. Fonction render_result: Affichage d'un Article

Objectif: Afficher un article de la CCN 3239 avec son titre, texte simplifié, texte officiel, et les liens vers les articles cités.

Fonctionnalités clés:

- Vérification de l'article: Si num_article est vide ou None, affiche un message d'erreur.
- Récupération des données: db.fetch_articles_complet(num_article).
- Affichage: Titre, texte simplifié (FR/EN), texte officiel (pliable).
- Détection des articles cités: Utilise une regex pour trouver les numéros d'articles cités.
- Chargement dynamique: container_lies pour les articles cités.

4. Fonction build_ui: Moteur Principal de l'Interface

Objectif: Générer dynamiquement l'interface utilisateur en fonction de l'étape actuelle, des choix de l'utilisateur, et de la langue.

Décorateur @ui.refreshable: Permet de rafraîchir l'UI sans recharger la page.

Fonction set_step:

- Met à jour state.step et state.choix.
- Mapping des métiers: Convertit les codes en libellés lisibles.
- Rafraîchissement: Appelle build_ui.refresh().

Zones:

- h_zone: Zone d'en-tête (sticky).
- c_zone: Zone de contenu (dynamique).

4.1. Étapes de build_ui

- Étape 0: Splash Screen (Écran d'Accueil)
- Étape 1: Sélection du Métier
- Étape 2: Sélection de la Situation (Gestion/Fin du Contrat)
- Étape 3: Sélection de la Famille
- Étape 4: Sélection du Thème
- Étape 5: Sélection de la Question
- Étape 6: Affichage du Résultat (Article)
- Recherche Directe (DIRECT)
- Liste des Annexes (LISTE_ANNEXES)
- Affichage d'une Annexe (VOIR_ANNEXE)

5. Page Principale (main_page)

Objectif: Point d'entrée de l'application (URL /).

Détails:

- ui.add_head_html: Configure le viewport, charge Font Awesome, injecte le CSS personnalisé, et ajoute un script de reconnexion.
- Initialisation: Crée un objet AppState, les zones h_zone et c_zone, et appelle build_ui.

6. Lancement du Serveur NiceGUI

Objectif: Configurer et lancer le serveur web.

Détails:

- host='0.0.0.0': Accessible depuis toutes les adresses IP.
- port: Utilise la variable d'environnement PORT (sinon 9000).
- storage_secret: Clé pour gérer les états utilisateurs.
- reload=False: Désactive le rechargement automatique.
- reconnect_timeout=3.0: Tolérance aux coupures réseau.
- show=False: N'affiche pas le message de lancement dans la console.

Analyse Globale et Points Forts

Architecture Modulaire:

- Séparation des responsabilités: AppState, render_result, build_ui, main_page.
- Code bien commenté.

Expérience Utilisateur (UX):

- Navigation intuitive.
- Recherche directe.
- Multilingue (FR/EN).
- Responsive et optimisé pour mobile.

- Interactivité: Chargement dynamique, sections pliables.

Gestion des Données:

- Base de données SQLite: Requêtes filtrées par métier.
- Fichiers statiques: Images, PDFs.

Optimisations Techniques:

- @ui.refreshable: Rafraîchissement partiel de l'UI.
- Gestion des états: Persistance des choix utilisateur.
- Reconnexion automatique: Script JavaScript pour gérer les coupures réseau.
- CSS personnalisé: Styles cohérents.

Points à Améliorer ou à Surveiller

1. Sécurité:

- storage_secret: Remplacer par une clé aléatoire et sécurisée.
- Injections SQL: Utiliser des requêtes paramétrées.

2. Performances:

- Connexions à la base de données: Utiliser une connexion unique ou un pool de connexions.
- Chargement des articles cités: Limiter la profondeur de récursion.

3. Maintenabilité:

- Duplication de code: Supprimer les doublons.
- Documentation: Ajouter un README pour les modules personnalisés.

4. Accessibilité:

- Contraste des couleurs: Respecter les normes WCAG.
- Navigation au clavier: Tester l'accessibilité.

5. Internationalisation (i18n):

- Textes statiques: Les ajouter dans UI_TEXT.

Diagramme de Flux de Navigation

Splash Screen (Étape 0)

|

Sélection du Métier (Étape 1)

|

Sélection de la Situation (Étape 2) -> [Gestion du Contrat / Fin du Contrat]

|

Sélection de la Famille (Étape 3)

|

Sélection du Thème (Étape 4)

|

Sélection de la Question (Étape 5)

|

Affichage de l'Article (Étape 6)

|

[Retour possible à chaque étape]

Autres chemins:

- Étape 1 -> Recherche Directe (DIRECT) -> Affichage de l'Article
- Étape 1 -> Liste des Annexes (LISTE_ANNEXES) -> Affichage d'une Annexe (VOIR_ANNEXE)